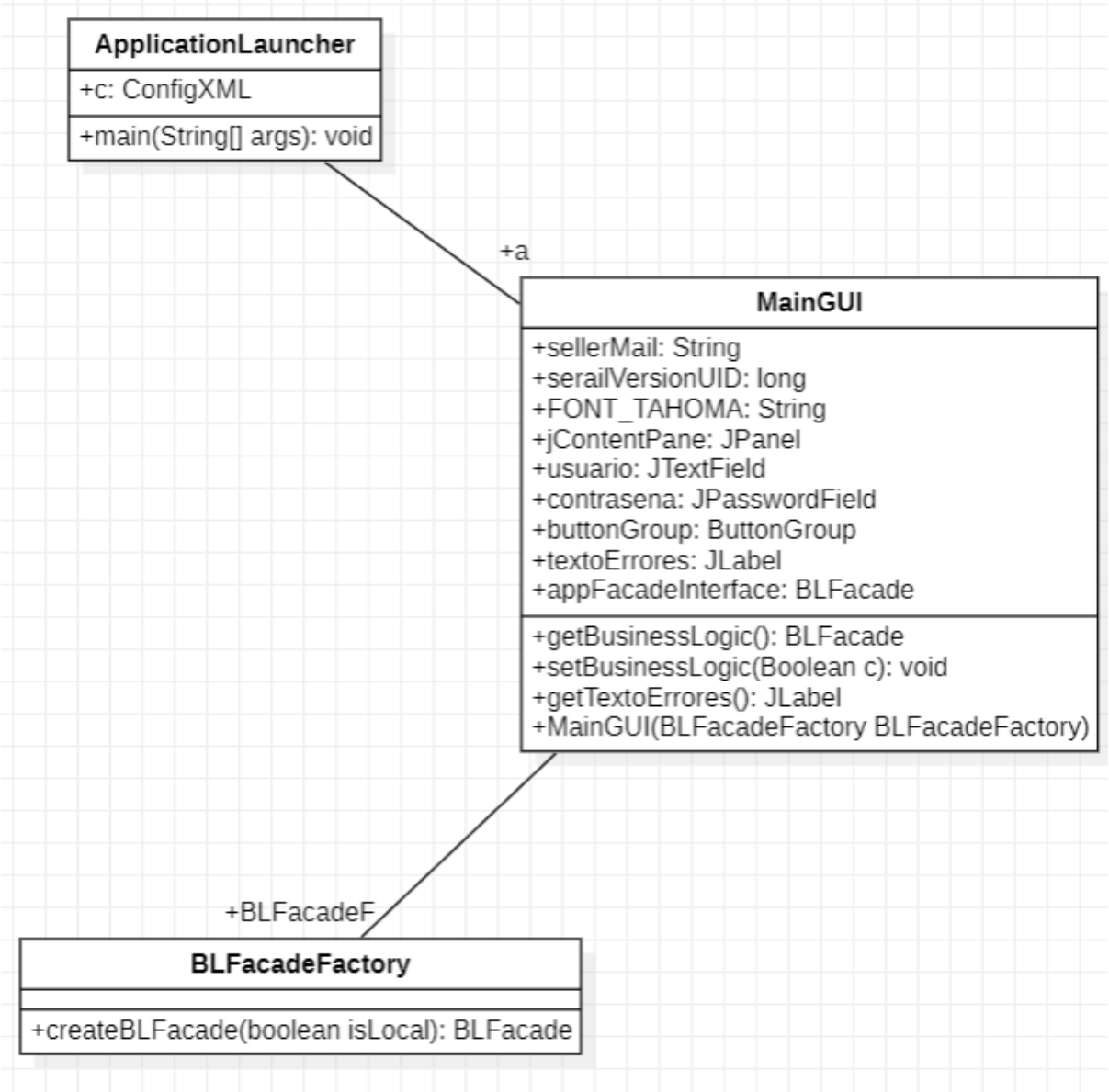
Factory

UML



Se han añadido las clases:

- **Creator**: `BLFacadeFactory`. Clase que se encarga de crear objetos BLFacade en función de si la clase ConfigXML es local o no; que se interpreta como el booleano isLocal que se le pasa.
- **Product**: El objeto que devuelve el método createBLFacade; que en este caso es un objeto BLFacade.

- **ConcreteProduct:** En función de si ConfigXML está definido para local o no, se creará un producto que siga una implementación concreta; pero que al final sigue siendo una lógica de negocio BLFacade.

Y se han editado las clases:

- `ApplicationLauncher`. Se ha eliminado el código que se encargaba de crear el *appfacadeinterface,* que después se le pasaba a MainGUI para que lo convirtiese en su lógica de negocio.
- `MainGUI`. Se le ha añadido un atributo *BLFacadeFactory* y un parámetro *BLFacadeFactory* a la constructora de la clase; de modo que cada vez que se cree se inicialice el atributo *BLFacadeFactory*. En la función setBusinessLogic se ha cambiado el parámetro de entrada de BLFacade a un Booleano y ahora la lógica de negocio se crea con el atributo *BLFacadeFactory*.

Código

En la clase BLFacadeFactory se crea un metodo createBLFacade(); al cual se le pasa un booleano que representa si ConfigXML es local o no; y en función de si es local o no creará un objeto BLFacade de una manera (Manera local) u otra (Manera remota); para después devolverlo.

```
public class BLFacadeFactory {

    public BLFacade createBLFacade(Boolean isLocal) throws MalformedURLException {
        BLFacade appFacadeInterface;

        if (isLocal) {
            DataAccess da= new DataAccess();
            appFacadeInterface=new BLFacadeImplementation(da);

        }
        else { //If remote
            ConfigXML c = ConfigXML.getInstance();
            String serviceName= "http://"+c.getBusinessLogicNode() +":"+ c.getBusinessLogicPort()+"/ws/"+c.getBusinessLogicName()+"?wsdl";
            URL url = new URL(serviceName);


            //1st argument refers to wsdl document above
            //2nd argument is service name, refer to wsdl document above
            QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");

            Service service = Service.create(url, qname);
            appFacadeInterface = service.getPort(BLFacade.class);
        }
        return (appFacadeInterface);
    }
}
```

En la clase MainGUI hemos añadido un parámetro BLFacadeFactory a su clase constructora y un atributo BLFacadeF del mismo tipo a la clase; de modo que cada vez que se cree una instancia de la misma se inicialice el atributo BLFacadeFactory dentro de la clase. Por otra parte, en la función setBusinessLogic; en vez de pasarle un BLFacade como parámetro directamente; ahora le pasamos un booleano que representa si la lógica de negocio de ConfigXML será local o no; y es el atributo BLFacadeF el que devuelve el objeto BLFacade a MainGUI.

```
43      private static BLFacade appFacadeInterface;
44      private static BLFacadeFactory BLFacadeF;
45
46      public static BLFacade getBusinessLogic(){
47          return appFacadeInterface;
48      }
49
50      public static void setBussinessLogic (Boolean c) throws MalformedURLException{
51          appFacadeInterface= BLFacadeF.createBLFacade(c);
52      }
53
54      public JLabel getTextoErrores() {
55          return textoErrores;
56      }
57
58      /**
59       * This is the default constructor
60       */
61      public MainGUI(BLFacadeFactory BLFacadeFactory) {
62          super();
63          BLFacadeF = BLFacadeFactory;
```

En la clase ApplicationLauncher se ha eliminado el código que creaba la lógica de negocio a pasarle a MainGUI, ya que ahora requiere de un booleano. Por ello, en la línea donde se define la lógica de negocio que usará MainGUI, le pasamos "ConfigXML.getInstance().getBusinessLogicLocal()"; y esa llamada a ConfigXML devuelve directamente un valor booleano que dice si ConfigXML es local o no, que permite a BLFacadeF en MainGUI establecer un BLFacade acorde al booleano que se le pasa.

```
30          try {
31              UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");
32
33              MainGUI.setBussinessLogic(c.isBusinessLogicLocal());
34              //ErreklamatuGUI g=new ErreklamatuGUI();
35              //g.setVisible(true);
```

Iterator

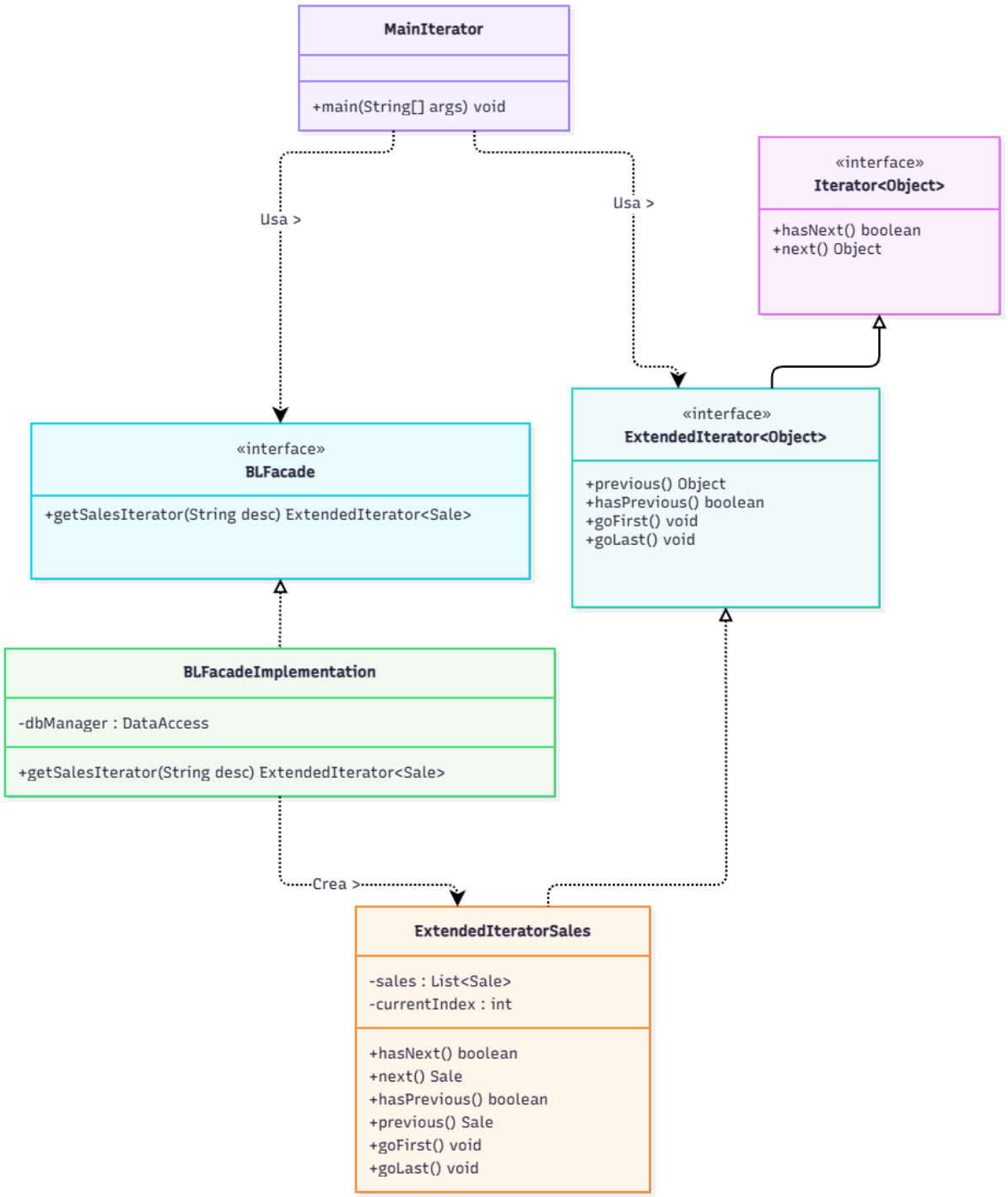
UML

Se han añadido las clases:

- **Iterator**: `ExtendedIterator`. Interfaz que amplía el iterador estándar para permitir navegación hacia atrás y saltos a los extremos.
- **ConcreteIterator**: `ExtendedIteratorSales`. Implementa la clase `ExtendedIterator`, almacenando la lista de ventas y controlando el índice actual para ejecutar los movimientos del recorrido.
- **Client**: `MainIterator`. Consume el iterador para visualizar las ofertas sin conocer qué estructura de datos (List, Vector, etc.) hay por debajo.

Y se han editado las clases:

- **Aggregate**: `BLFacade`. **Se ha añadido la función** `getSalesIterator(String desc)`, **la cual define.**
- **ConcreteAggregate**: `BLFacadeImplementation`. **Se ha añadido la función** `getSalesIterator(String desc)`, **la cual implementa. Su función es instanciar el iterador entregando los datos obtenidos de la base de datos.**



Código

ExtendedIteratorSales

Es la clase responsable de aislar los datos y controlar el recorrido. Utiliza una variable de estado (`currentIndex`) como cursor para gestionar la navegación: avance (`currentIndex++`), retroceso (`--currentIndex`) y saltos a los límites de la colección (`goFirst, goLast`).

```java
public class ExtendedIteratorSales implements ExtendedIterator<Sale> {
    private List<Sale> sales;  5 usages
    private int currentIndex;  7 usages

    public ExtendedIteratorSales(List<Sale> sales) {  1 usage  anelric
        this.sales = sales;
        this.currentIndex = 0;
    }

    @Override  anelric
    public boolean hasNext() { return currentIndex < sales.size(); }

    @Override  anelric
    public Sale next() {
        if (!hasNext()) {
            throw new IndexOutOfBoundsException("No more elements");
        }
        return sales.get(currentIndex++);
    }
```

```
    @Override  1 usage  anelric
    public Sale previous() {
        if (!hasPrevious()) {
            throw new IndexOutOfBoundsException("No previous elements");
        }
        return sales.get(--currentIndex);
    }

    @Override  2 usages  anelric
    public boolean hasPrevious() { return currentIndex > 0; }

    @Override  1 usage  anelric
    public void goFirst() { currentIndex = 0; }

    @Override  1 usage  anelric
    public void goLast() {
        currentIndex = sales.size();
    }
}
```

BLFacade

Añadimos el método `getSalesIterator` para definir el contrato público. Permite a los clientes (como la interfaz gráfica) solicitar los datos listos para recorrer, ocultando por completo qué estructura técnica (List, Vector, etc.) se utiliza internamente.

```
@WebMethod public ExtendedIterator<Sale> getSalesIterator(String desc);
```

BLFacadeImplementation

Implementa el método obteniendo las ofertas de la base de datos y encapsulándolas instanciando el iterador. Se hace así para no devolver la lista cruda al cliente, aislando la capa de datos.

```
public ExtendedIterator<Sale> getSalesIterator(String desc) {
    dbManager.open();
    List<Sale>  rides=dbManager.getSales(desc);
    dbManager.close();
    return new ExtendedIteratorSales(rides);
}
```

Ejecución

```
----------------------
FROM    LAST    TO  FIRST
9;sukaldeko mahaia;45.0
8;polar kilor erlojua;30.0
7;orbea mendiko bizikleta;225.0
6;imac 27;200.0
5;samsung 42" telebista;175.0
4;salomon mendiko botak;20.0
3;futbol baloia;10.0


----------------------
FROM    FIRST   TO  LAST
3;futbol baloia;10.0
4;salomon mendiko botak;20.0
5;samsung 42" telebista;175.0
6;imac 27;200.0
7;orbea mendiko bizikleta;225.0
8;polar kilor erlojua;30.0
9;sukaldeko mahaia;45.0

Process finished with exit code 0
```

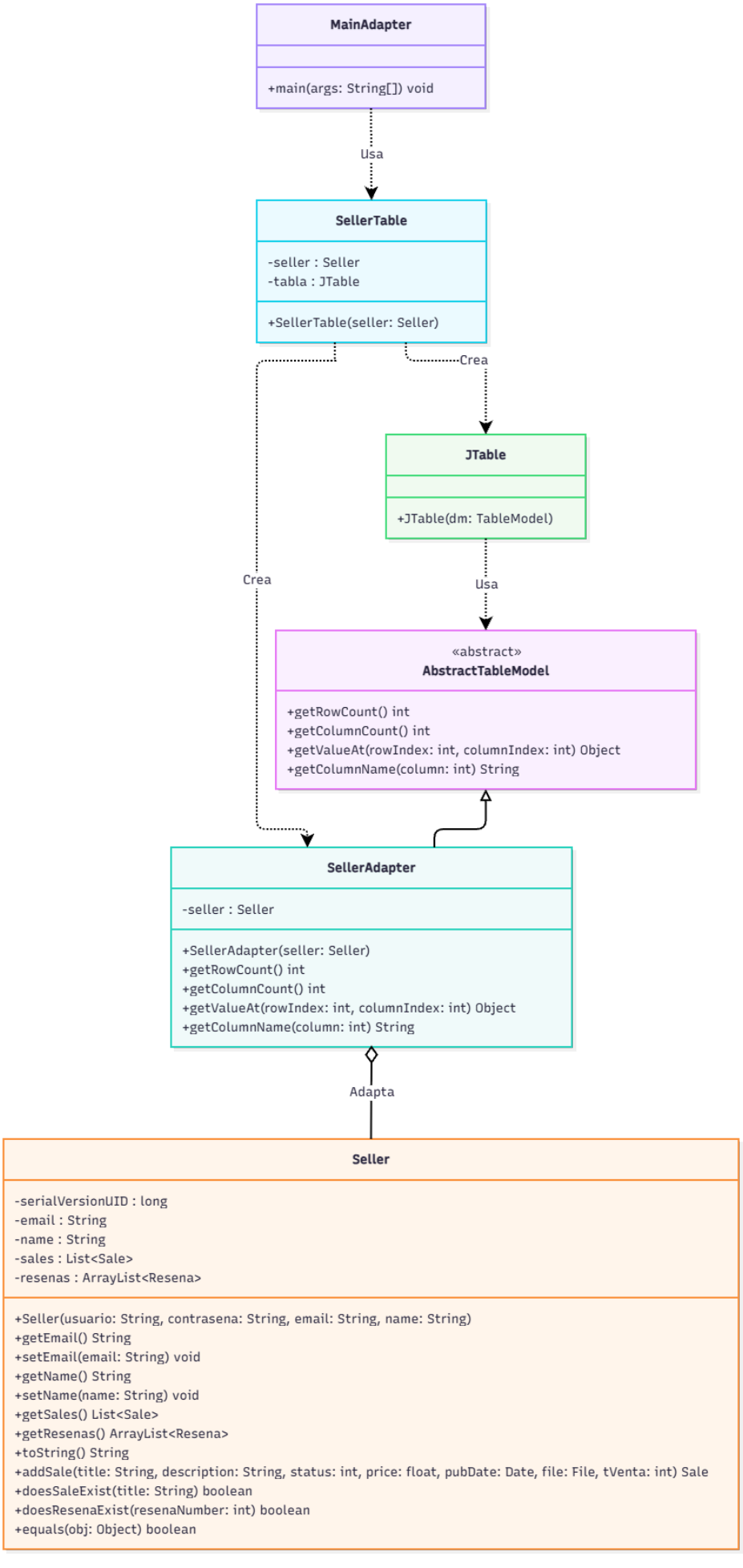
Adapter

UML

- **Target** (Objetivo): `AbstractTableModel`. **Es la clase abstracta (o interfaz) que define los métodos que el** `JTable` **espera recibir para poder dibujar una tabla (**`getRowCount`, `getColumnCount`, `getValueAt`**).**
- **Adapter** (Adaptador): `SellerAdapter`. **Hereda de** `AbstractTableModel` **y actúa como puente. Traduce los datos de la clase** `Seller` **al formato de filas y columnas requerido por la tabla.**

Además, se han añadido las clases:

- **Adaptee** (Adaptado): `Seller`. **Es el objeto de nuestro dominio que contiene la información real (la lista de** `Sale`**), pero cuya interfaz original no es compatible de forma directa con un JTable.**
- **Client** (Cliente): `SellerTable / JTable`. **Es el componente visual que consume el adaptador pensando que es un modelo de tabla estándar, sin importarle que internamente esté leyendo de un objeto** `Seller`.


MainAdapter
+main(args: String[]) void
Usa
SellerTable
-seller : Seller
-tabla : JTable
+SellerTable(seller: Seller)
Crea
Crea
JTable
+JTable(dm: TableModel)
Usa
«abstract»
AbstractTableModel
+getRowCount() int
+getColumnCount() int
+getValueAt(rowIndex: int, columnIndex: int) Object
+getColumnName(column: int) String
SellerAdapter
-seller : Seller
+SellerAdapter(seller: Seller)
+getRowCount() int
+getColumnCount() int
+getValueAt(rowIndex: int, columnIndex: int) Object
+getColumnName(column: int) String
Adapta
Seller
-serialVersionUID : long
-email : String
-name : String
-sales : List<Sale>
-resenas : ArrayList<Resena>
+Seller(usuario: String, contrasena: String, email: String, name: String)
+getEmail() String
+setEmail(email: String) void
+getName() String
+setName(name: String) void
+getSales() List<Sale>
+getResenas() ArrayList<Resena>
+toString() String
+addSale(title: String, description: String, status: int, price: float, pubDate: Date, file: File, tVenta: int) Sale
+doesSaleExist(title: String) boolean
+doesResenaExist(resenaNumber: int) boolean
+equals(obj: Object) boolean

Código

SellerAdapter

Hereda de `AbstractTableModel` y guarda el objeto `Seller` internamente. Actúa como puente traductor sobrescribiendo los métodos obligatorios (`getRowCount`, `getValueAt`) para transformar la lista unidimensional de ventas en la matriz de filas y columnas que Java Swing necesita.

```java
public class SellerAdapter extends AbstractTableModel {  2 usages
    private Seller seller;  3 usages

    public SellerAdapter(Seller seller) { this.seller = seller; }

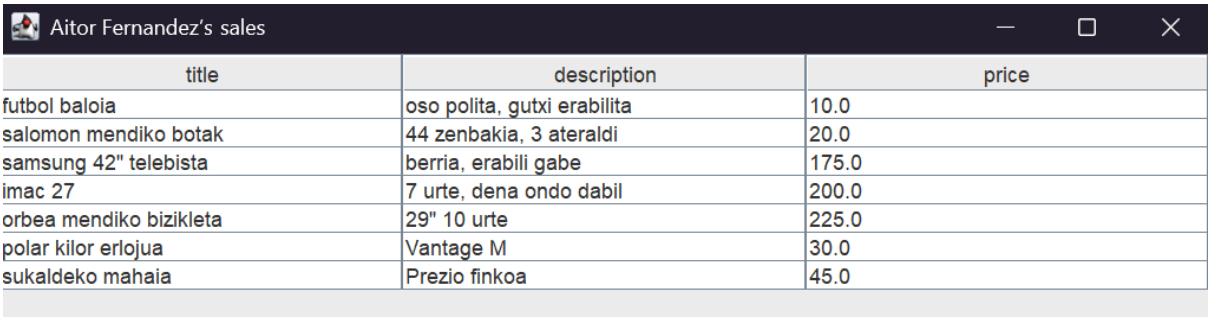
    @Override  anelric
    public int getRowCount() { return seller.getSales().size(); }

    @Override  anelric
    public int getColumnCount() {
        return 3;
    }

    @Override  anelric
    public Object getValueAt(int rowIndex, int columnIndex) {
        Sale sale = seller.getSales().get(rowIndex);
        switch (columnIndex) {
            case 0:
                return sale.getTitle();
            case 1:
                return sale.getDescription();
            case 2:
                return sale.getPrice();
            default:
                return null;
        }
    }
```

```java
    @Override  anelric
    public String getColumnName(int column) {
        switch (column) {
            case 0:
                return "title";
            case 1:
                return "description";
            case 2:
                return "price";
            default:
                return super.getColumnName(column);
        }
    }
}
```

Ejecución



title	description	price
futbol baloia	oso polita, gutxi erabilita	10.0
salomon mendiko botak	44 zenbakia, 3 ateraldi	20.0
samsung 42" telebista	berria, erabili gabe	175.0
imac 27	7 urte, dena ondo dabil	200.0
orbea mendiko bizikleta	29" 10 urte	225.0
polar kilor erlojua	Vantage M	30.0
sukaldeko mahaia	Prezio finkoa	45.0